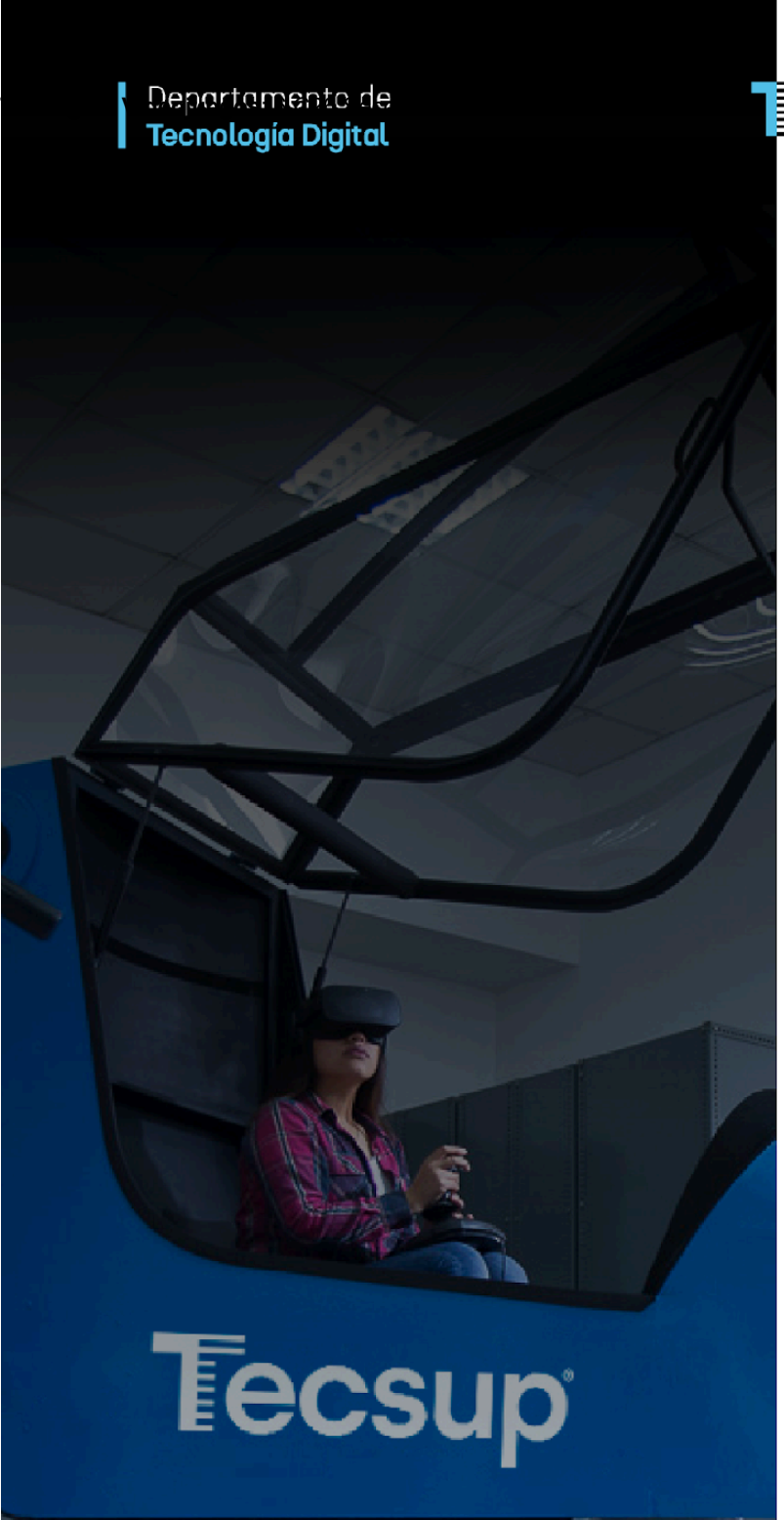
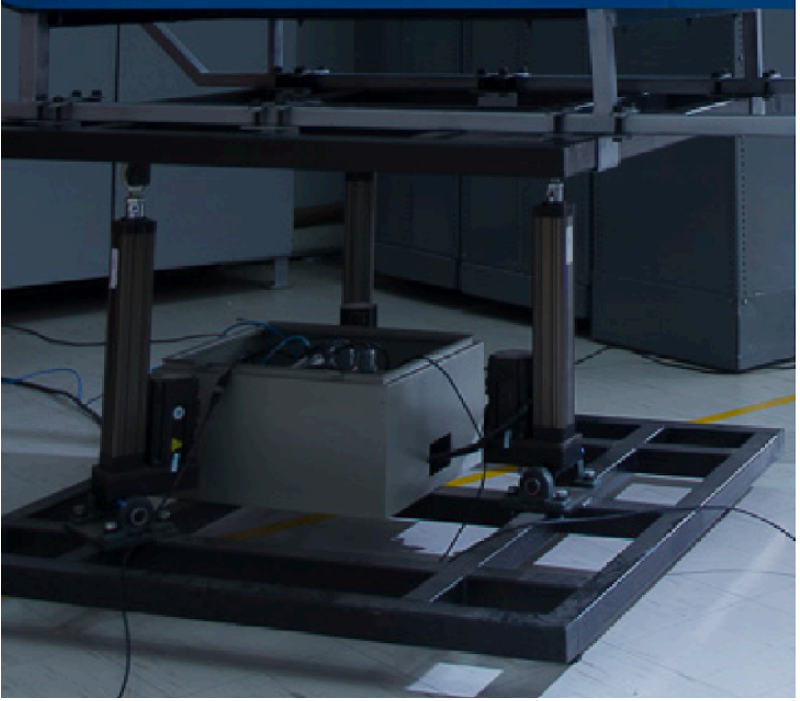
Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO





LABORATORIO N° 03

Diseño y Desarrollo de Simuladores y Videojuegos

DOCENTE:

Bestard Aroche, Yunior

CURSO:

Desarrollo de videojuegos móviles avanzado
6 - C26 - Sección A

TESTING DE VIDEOJUEGOS: Quality Assurance y Quality Control

I. Instrucción de entrega

Complete sus respuestas directamente en este mismo documento, debajo de cada ejercicio. Al finalizar, exporte el archivo completo (enunciados y respuestas) en formato PDF y súbalo con el nombre GLAB-S03-Apellido-Nombre.pdf. No se aceptan documentos separados de enunciado y respuesta.

II. Capacidades

- **Definir y diferenciar los conceptos de Quality Assurance (QA) y Quality Control (QC)**, aplicándolos al proceso de desarrollo de videojuegos para garantizar estándares de calidad.
- **Aplicar técnicas de aseguramiento de calidad (QA)** en proyectos de videojuegos, estableciendo políticas, checklists y métricas preventivas que reduzcan la aparición de errores en las distintas etapas de Scrum.
- **Diseñar y ejecutar pruebas de control de calidad (QC)** en prototipos de videojuegos, documentando casos de prueba, resultados y correcciones necesarias para validar el cumplimiento de requisitos funcionales y técnicos.
- **Integrar QA y QC en la gestión ágil de proyectos (Scrum)**, vinculando las tareas documentadas e implementadas con procesos de validación y aseguramiento de calidad, para optimizar la iteración del producto en desarrollo.

III. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

IV. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

V. Normas empleadas

- *No aplica*

VI. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.

VII. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual en caso de grupo de tesis en equipo

VIII. Procedimiento

Ejercicio 1 – QA aplicado a su proyecto

Elabora un **checklist de aseguramiento de calidad (QA)** para tu prototipo de videojuego, considerando:

- Rendimiento (FPS mínimo, memoria).
- Jugabilidad (fluidez de controles, claridad de feedback visual/sonoro).
- Compatibilidad (plataformas y dispositivos definidos en tu tesis).
  Entrega: tabla con al menos **5 ítems de QA** y el estado actual de cumplimiento.

Ejemplo: *En tu tabla podrías incluir la fila 'FPS mínimo en dispositivo objetivo — estándar: ≥30 FPS sostenidos — cumple: No'.*

Rendimiento

Estándar de referencia: ≥ 30 FPS sostenidos en el PC objetivo.

ID	Ítem de QA	Estándar (criterio de aceptación)	Método de verificación	Estado	Evidencia / Hallazgo
R-01	FPS sostenidos en el PC objetivo	≥ 30 FPS durante 60 s de gameplay continuo	Game view › Stats (FPS y frame time), 3 repeticiones	Pendiente	Sin medir: requiere build de Window
R-02	Frame time en percentil 95	< 33,3 ms	Profiler › CPU › Main Thread durante 60 s	Pendiente	Sin medir
R-03	Ausencia de picos por GC / asignación	ningún frame por encima de 50 ms	Profiler › GC Alloc, sesión completa	Pendiente	Sin medir
R-04	Memoria gestionada estable	< 250 MB y delta < 5 MB en 10 min	Profiler › Memory, capturas a 0 / 5 / 10 min	Pendiente	Sin medir
R-05	Sin fuga por spawn/destroy de puertas	delta aproximado 0 tras 20 niveles fallidos	Profiler › Memory antes y después del ciclo	Pendiente	Sin medir
R-06	Draw calls dentro de presupuesto	< 300 draw calls	Profiler › Rendering, escena Gameplay	Pendiente	RIESGO: 226 MeshRenderer en una sola escena
R-07	Tiempo de carga Login → Gameplay	< 5 s	Cronómetro sobre la build	Pendiente	El proyecto nunca se ha compilado
R-08	Presupuesto de memoria de texturas	< 200 MB en runtime	Profiler › Memory › Textures	Pendiente	RIESGO: 106,4 MB de origen, 61 texturas a 4096 px

Jugabilidad

Cubre controles y claridad de feedback visual y sonoro, según exige la consigna.

ID	Ítem de QA	Estándar (criterio de aceptación)	Método de verificación	Estado	Evidencia / Hallazgo
J-01	Respuesta de movimiento	< 1 frame entre input y desplazamiento	Manual: recorrido recto de 10 s	No	PlayerLook.sensitivity = 0,3 frente a 2,0 por defecto: la respuesta es muy lenta
J-02	Cámara sin tirones ni pitch flip	límites de ±80 grados respetados, sin saltos	Manual: giro vertical completo y rápido	Pendiente	Sin medir
J-03	Prompt de interacción visible al enfocar	aparece en < 200 ms al mirar el objeto	Manual: enfocar y alejarse de cada puerta	No	CORREGIDO: el prompt ya no comparte canal con los mensajes críticos (FeedbackUI.ShowPrompt)
J-04	Feedback visual de escaneo	progreso y resultado visibles durante el escaneo	Manual: escanear una puerta segura y una insegura	No	CORREGIDO: guardas null en ScannerSystem; la escena de escaneo ya funcionaba

ID	Ítem de QA	Estándar (criterio de aceptación)	Método de verificación	Estado	Evidencia / Hallazgo
J-05	Feedback sonoro por evento crítico	≥ 1 cue en escaneo, puerta, éxito y error	Manual: recorrer los cuatro eventos	No	El proyecto no contiene NINGÚN AudioSource ni clip de audio (0 asset
J-06	La regla de 3 intentos no es evadible	3 fallos obligan a volver al login, sin bypass	Intento de evasión: fallar 3 veces y volver a entrar	No	StrikeManager se reinicia al recargar escena: los intentos son ilimitados si se repite el login (H-6)
J-07	Los 4 niveles son resolubles	4 de 4 con puerta correcta alcanzable	Play-through de los cuatro niveles	No	CORREGIDO: el nivel 4 no contenía ninguna puerta VerySafe (H-7)
J-08	Reinicio tras fallo sin estado residual	checkpoint, timer e integridad reseteados	Manual: fallar un nivel y reintentar	Pendiente	Sin medir
J-09	El login no es evadible	es imposible entrar a Gameplay sin validarse	Abrir Gameplay 1.unity directamente en el Editor	No	La escena se puede abrir sin pasar po el login durante el desarrollo

Compatibilidad

Alcance: PC Windows standalone. Android excluido por decisión de alcance.

ID	Ítem de QA	Estándar (criterio de aceptación)	Método de verificación	Estado	Evidencia / Hallazgo
C-01	Build Windows x64 compila y arranca	sin errores de compilación	Build Settings › Build	No	El proyecto nunca se ha compilado: r existe carpeta Build ni ejecutable
C-02	Resolución mínima soportada definida	1024 × 768	ProjectSettings › Player	Sí	Configurado en defaultScreenWidth defaultScreenHeight
C-03	Perfil de calidad por plataforma	PC activo y Mobile definido	QualitySettings.asset	Sí	Ambos perfiles existen; vSync 0 en lo dos
C-04	Teclado y ratón funcionales	WASD / flechas, E, F, Tab y Esc	Manual: cada acción por separado	Sí	Sin incidencias detectadas
C-05	Cero excepciones en una sesión completa	0 excepciones	Consola de la build, sesión completa	No	Corregidas 17 llamadas a FeedbackU persisten singletons accessados vía Instance sin guard
C-06	Sin pérdida de referencias entre escenas	Login → Gameplay → Login sin errores	Manual: ciclo completo de escenas	Pendiente	Sin medir
C-07	Build Android	—	—	No aplica	FUERA DE ALCANCE: escalabilidad futura según la tesis
C-08	Respuesta al redimensionado de ventana	sin artefactos ni cámara rota	Manual: arrastrar los bordes de la ventana	Pendiente	Sin medir

Ejercicio 2 – QC con tareas de Scrum

Selecciona **3 tareas implementadas en tu backlog Scrum** (por ejemplo: un sistema de colisiones, inventario o IA enemiga). Para cada una, diseña y ejecuta un **caso de prueba (QC)** con la misma estructura vista en clase:

- ID y requisito o historia relacionada.
- Objetivo.
- Build, dispositivo y entorno.

- Precondiciones y datos de entrada.
- Pasos numerados.
- Resultado esperado.
- Resultado obtenido.
- Estado (aprobado / fallido) y evidencia.
- Ejecuta cada test y registra los resultados.

Entrega: documento con los 3 casos de prueba ejecutados (uno por tarea), en el formato anterior.

Ejemplo: *TC-INV-002 — Historia: 'Como jugador quiero recoger objetos del suelo'. Build 0.4.2, Samsung S6. Precondición: inventario vacío, un objeto en escena. Pasos: acercarse al objeto y pulsar recoger. Esperado: el objeto aparece en el inventario y desaparece del suelo. Obtenido: el objeto aparece duplicado. Estado: Fallido. Evidencia: video adjunto.*

Tareas de backlog seleccionadas

- T2. Sistema de interacción por raycast del jugador (colisiones de interacción)
- T4. Escáner de evidencia en la tablet (recolección de información)
- T6. Reglas de estado del minijuego (strikes / integridad / reingreso)

TC-INT-001 — Interacción por raycast (sistema de colisiones de interacción)

ID / Historia: TC-INT-001 — "Como jugador quiero apuntar a la consola y a las puertas para interactuar al alcance" (T2).

Objetivo: Verificar que el rayo del jugador detecta interactuables dentro de interactDistance (3 m), muestra su prompt y ejecuta la interacción con E.

Build, dispositivo y entorno: Editor Unity 6000.3.9f1, PC Windows, Game View 1920×1080 (simulación; sin build de player).

Precondiciones y datos de entrada: Escena Gameplay.unity; PlayerInteractor con interactDistance = 3, interactableLayers asignado y cámara; VerificationConsole con VerificationManager referenciado y collider en la layer de interacción; partida con 0 strikes.

Pasos:

Entrar a Play.

Acercarse a la consola a < 3 m y apuntar al collider.

Observar el prompt de interacción en FeedbackUI.

Pulsar E.

Rotar la cámara para salir del rango de la consola (sin pulsar nada).

Observar el HUD (prompt, HUD de verificación, tablet, timer).

Resultado esperado: (3) Prompt "Presiona E para iniciar la verificación." visible; (4) verificación inicia: HUD de verificación, tablet y timer activos; (5) al salir del rango el prompt desaparece; (6) estado estable.

Resultado obtenido: Pasos 3 y 4 OK. Paso 5 FALLA: el prompt "Presiona E…" permanece en pantalla al salir del rango. Causa en código: PlayerInteractor.CheckForInteractable() limpia la tablet en la transición a null pero no oculta el mensaje de FeedbackUI (PlayerInteractor.cs:83–88).

Estado: FALLIDO (parcial — interacción correcta, limpieza de prompt rota).

Evidencia: captura del HUD con prompt persistente tras paso 5; referencia PlayerInteractor.cs:83-88 (BUG-005 del reporte QA).

TC-SCN-002 — Escáner de evidencia en la tablet

ID / Historia: TC-SCN-002 — "Como jugador quiero escanear una puerta y ver su resultado de seguridad en la tablet" (T4).

Objetivo: Verificar que mantener F escanea una DoorOption, muestra "Escaneando…" → resultado correcto en la tablet, bloquea el movimiento mientras dura y que no coexistan acciones simultáneas con la verificación.

Build, dispositivo y entorno: Editor Unity 6000.3.9f1, PC Windows, Game View (simulación).

Precondiciones y datos de entrada: ScannerSystem en escena con scanDuration = 3 s; una DoorOption a 2 m; tabla habilitada; partida en modo exploración.

Pasos:

Entrar a Play junto a una puerta.

Mantener F sobre la puerta.

Observar tablet "Escaneando…" y bloqueo de movimiento.

Soltar F y esperar 3 s → resultado SEGURO/INSEGURO.

Repetir mantenido F y, sin soltarlo, pulsar E.

Resultado esperado: Pasos 1–4: escaneo con resultado coherente con isCorrectDoor y movimiento re-habilitado; paso 5: el jugador no podría iniciar una segunda acción miendo el escaneo sigue, o el escaneo se cancela (una acción activa a la vez).

Resultado obtenido: Pasos 1–4 OK (resultado correcto, movimiento bloqueado/liberado). Paso 5 FALLA: al pulsar E durante el escaneo, StartVerification() se ejecuta mientras ScanRoutine sigue vivo → verificationActive == true y isScanning == true simultáneamente; la tablet recibe el resultado del escaneo y a la vez la verificación la reasigna (VerificationManager.cs:62; ScannerSystem.cs:56–86). Reproducción ~70%.

Estado: FALLIDO (concurrencia de estados).

Evidencia: reproducción con F y E simultáneos; referencia BUG-003 (no hay guard en PlayerInteractor.OnInteract para escaneo activo).

TC-VER-003 — Reglas de estado: límite de intentos y reingreso

ID / Historia: TC-VER-003 — "Como sistema quiero bloquear la verificación tras 3 errores e impedir el reingreso" (T6).

Objetivo: Verificar que al alcanzar maxStrikes = 3 el jugador es devuelto al login y no puede reingresar a la verificación reintentando.

Build, dispositivo y entorno: Editor Unity 6000.3.9f1, PC Windows (simulación).

Precondiciones y datos de entrada: StrikeManager (max 3), IntegritySystem (damagePerMistake = 25, clamp 0), TimerSystem (startTime = 30 s); 3 niveles configurados; consola activa.

Pasos:

Iniciar la verificación con E en la consola.

Elegir 3 puertas incorrectas (integridad 100 → 75 → 50 → 25; en la 3ª granja real 3 strikes).

Observar mensajes "Acceso denegado", integridad descendente y al 3er fallo "Regresando al login" con vuelta al lobby.

Verificar que el límite (3/3) bloquea nuevas verificaciones.

Resultado esperado: Tras 3/3 strikes el acceso queda bloqueado; la consola no permite reiniciar la verificación; el castigo (integridad 0 / strikes) persiste.

Resultado obtenido: Pasos 1–3 OK (mensajes, integridad por debajo no negativa, vuelta al login). Paso 4 FALLA: la consola se reactiva (FailVerification → consoleObject.SetActive(true), VerificationManager.cs:225) y pulsar E llama StartVerification(), que no valida HasReachedMaxStrikes() (VerificationManager.cs:45–94) y resetea integridad y avance → reingreso indefinido posible con el castigo anulado. Los strikes no se resetean pero tampoco bloquean el reingreso.

Estado: FALLIDO (regla de juego evadible).

Evidencia: secuencia de capturas: 3 fallos → login → pulsar E → verificación reiniciada con integridad 100; referencia BUG-006.

Ejercicio 3 – QA/QC combinado

Identifica una **tarea pendiente en tu backlog**.

Define el criterio de aceptación de esa tarea (¿qué debe cumplir para considerarse terminada?).

1. Propón una acción de **QA** para prevenir errores durante su implementación (ej. revisión de código, estándares de naming, pruebas unitarias obligatorias).
2. Diseña un **test de QC** que permita validar la tarea una vez implementada, verificando que cumple el criterio de aceptación definido.

Entrega: un cuadro comparativo (criterio de aceptación / QA / QC) aplicado a esa tarea.

Ejemplo: *Criterio de aceptación: 'el dash no debe atravesar paredes'. QA: revisión de código del sistema de colisiones antes del merge. QC: ejecutar el dash contra 3 tipos de pared y verificar que el personaje se detiene.*

Criterio de aceptación	QA (prevención)	QC (validación)
E en consola con 3/3 no inicia verificación	Test unitario obligatorio StartVerification_IsRejected_WhenMaxStrikesReached (falla antes del fix)	TC-VER-004: fallar 3 puertas → pulsar E → NO arranca

Consola no interactuable al límite	Code review + grep de todos los sets de verificationActive = true	Verificar consola inerte y mensaje de bloqueo
Castigo de integridad (0%) persiste tras reintento	No resetear CurrentIntegrity sin pasar la regla	Integridad sigue a 0% tras pulsar E
Reingreso imposible (0 veces)	Guard con naming de regla (HasReachedMaxStrikes) al inicio de StartVerification	3 repeticiones del ciclo fialoan → ninguna reentrada
Suite regrescinda	Las 10 pruebas EditMode existentes en verde (gate)	Run All 10/10 + test nuevo verde

Ejercicio 4 – Retroalimentación cruzada

En parejas, intercambien sus **checklists de QA y casos de prueba QC**.

- Cada estudiante debe **ejecutar un test** en el prototipo de su compañero.
- Si el test falla, documenta la falla como **reporte de defecto** (título específico, build/entorno, pasos de reproducción, resultado esperado y actual, severidad y prioridad).
- Registra hallazgos y propón mejoras en el proceso (QA) y en el producto (QC).

Entrega: breve reporte comparando su percepción con los resultados del compañero, incluyendo al menos un reporte de defecto (si se encontró alguno) o una justificación de por qué no se encontraron fallas.

Ejemplo: *Título de defecto: 'El jugador pierde la munición equipada al pausar y reanudar la partida' (severidad: media, prioridad: alta).*

Ejercicio 5 – Conclusión reflexiva

Cree un video explicando los ejercicios realizados en esta práctica y respondiendo:

- ¿Cómo mejoró la integración de QA y QC la visión de calidad en tu prototipo de tesis?
- ¿Qué prácticas llevarás a la siguiente iteración de tu proyecto?

Ejemplo: *'Antes de este laboratorio no revisaba builds en dispositivo real; el checklist de QA me hizo detectar que el juego bajaba de 30 FPS en el nivel 2. Para la siguiente iteración voy a agregar ese ítem a mi Definition of Done.'*

Link:

https://youtu.be/BIWugh8IIOg

Conclusiones: